AI Tools: Introduction to Claude Code

Xugan Chen

Yale School of Management

MAM Orientation BootCamp

August 27, 2026

About me

- **Xugan Chen** – PhD candidate, Financial Economics, Yale SOM. At Yale since 2020 (predoc 2020–2022, PhD since 2022).
- **TA for:** Entrepreneurial Finance; Venture Capital and Private Equity; Statistical Foundations; Behavioral Finance.
- **Research:** innovation; AI in finance and economics; corporate finance; entrepreneurial finance.

Feedback is welcome – now, or any time after today.

Getting to know you

Show of hands. Which side of each pair are you here for?

Public markets	vs	**Private markets**
Industry	vs	**Academia**
Quant research	vs	**Fundamental research**
Equities	vs	**Other asset classes**

Today's task sits in **private markets**, and the workflow is the same whichever side you raised your hand on.

Getting to know you: How do you use AI tools?

Tool	What you actually do
Browser chat, inline completion	Copy code out, paste it in, run it, paste the error back
Agentic IDE (Cursor)	Tell it what to do; it edits files inside the editor
Terminal agent (Claude Code, Codex)	**Full environment**: reads, writes, runs commands, plans
Level 3 plus MCP tools	Wire it to databases, APIs, other services
Agent in a container	One instruction, ninety unattended minutes
Your own model	Build and train it

Today is the terminal agent, with one look at agents in a container at the end. Agent in a container is not better; it is more setup.

Ladder adapted from Kyle Jensen.

Working with AI, one year ago

- A **chat window**. You describe the problem in prose, it hands back a snippet, you paste the snippet.
- It could not see your data, your files, or your errors. You were the clipboard between the model and the project.
- The code was **roughly right and often not runnable**, which meant you still had to be the one who ran it.

That is the version most people still have in mind.

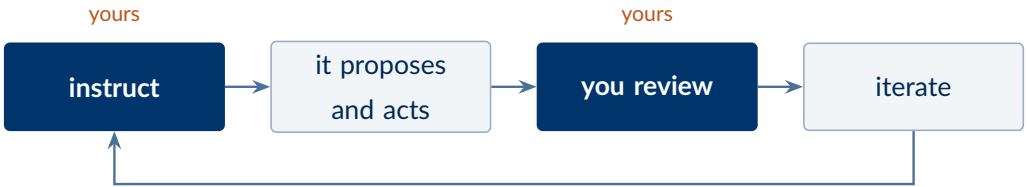
Working with AI, now: it acts in your project

An agent that lives in your terminal, inside your project directory.

- **Reads and writes files** in the repo – your data, your code, your notes.
- **Runs commands** – installs packages, executes scripts, reads the errors.
- **Iterates** – sees its own output fail and tries again, without you retyping.

It does not hand you a snippet to paste. It acts in the project and **lives with the consequences.**

The loop is easy. Instruct and review is the job.

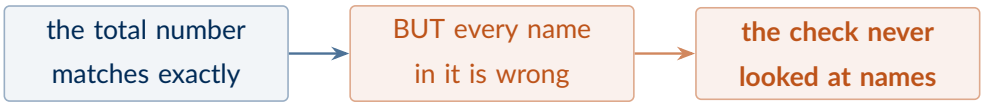


- The two boxes with your name on them are **the entire job**.
- Everything we add today – planning, verification, generalization – makes the instruction sharper, or makes review possible on output **too large to read**.

Today you will sign off on a dataset of 37,197 rows without reading one of them.

Why reviewing is hard: it checks its own work

- Ask it to build a dataset and you usually get a check you did not ask for: "I reconciled the total to the reported figure – it matches."
- That is genuinely useful. It is also the trap, because **a check looks at one thing**, and the agent decided which thing.



A check that passes tells you about **the part it looked at**. It says nothing about the rest.

You will meet this four times this afternoon, and it looks different every time.

Today's objective

- **Not** a feature walkthrough – the menu of options will have changed by October.
- **Not** an intro to programming – most of you can already code.
- **Is yours to build**, with the agent, on your own machine – not a demo you watch.
- **Is** practice at **checking output you cannot read** – tens of thousands of rows.
- **Is** practice at **patterns that move** to the rest of your work, not just to this task.

It is never **"can it write the code"**. It is **"how do I know this is right"** and **"how do I scale it"**.

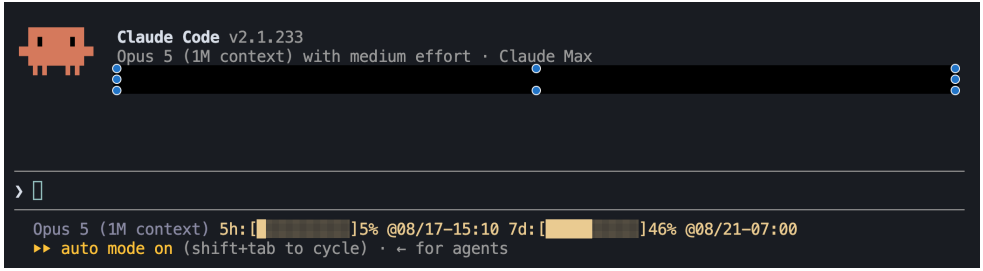
The roadmap

Block	Mode
1. Introduction	lecture
2. Applied AM project: a private credit dataset from SEC filings	demo, then hands-on
Break, 10 minutes – halfway through the project	
...*the project continues*: generalize, then analyse	hands-on
3. More useful concepts and tips	lecture
4. Advanced usage examples	demo and Q&A

Half the session (in Part 2) is hands-on. So have your laptop out and Claude Code installed. Work on your own or with the person next to you – either is fine.

Basic setup

- **Install** it, either way:
  - With npm, if you have Node.js: `npm install -g @anthropic-ai/claude-code`
  - Or the standalone installer, for Mac, Linux and Windows: `code.claude.com/docs`
- **Then** `cd` into a project folder and type `claude`. That is all.

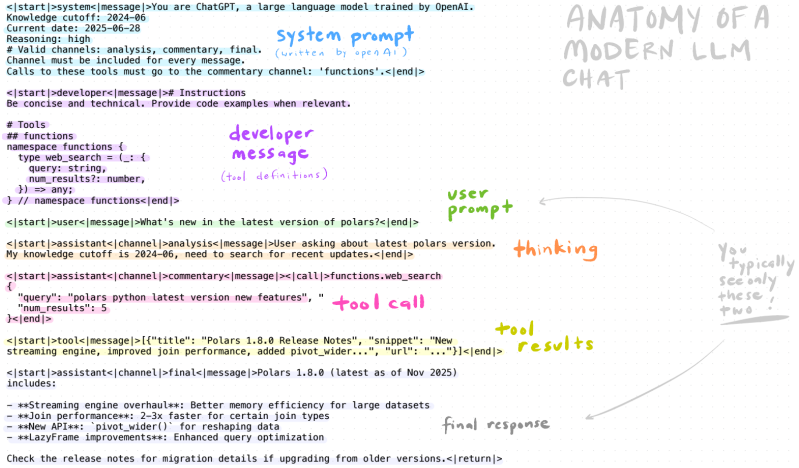


If your terminal does not look like this, fix it now. The rest of the session assumes a working install.

Tokens: the unit everything is measured in

- **Tokens** are the pieces text is chopped into before the model sees it.
  - The short sentence “I love cats” is broken into **3 tokens** (I, love, cats).
  - A longer word like “unbreakable” may be split into **2 tokens** (un and breakable).
  - Rule of thumb: in English, **1 token is about 4 characters, or 0.75 words**.
- Claude Fable/Opus/Sonnet: a **1M-token context window**, roughly 750,000 words.
- Later we will work on SEC filings – **one filing runs 10 to 25 MB of HTML**.
  - At 25 MB that is roughly **6 million tokens – six full context windows**.
  - **So nothing ever reads the whole document.** It gets searched, sliced, and read a slice at a time. You will watch that happen this afternoon.

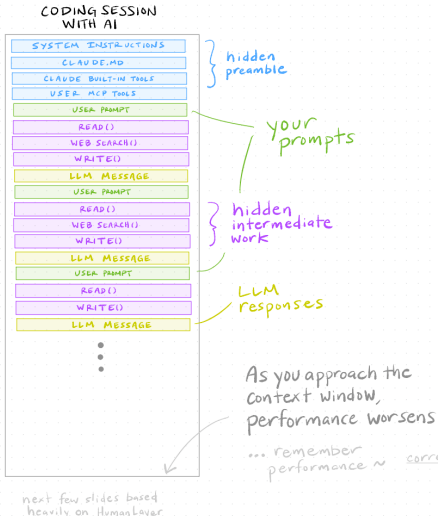
Context window: You see two things, but the model sees all of it.



A “chat” is one long document passed back and forth. System prompt, tool definitions, thinking, tool calls, results – **all of it counts against your window**, and you see almost none of it.

Source: A Series on Claude Code by Paul Goldsmith-Pinkham.

Context window: Long sessions degrade, so plan for it.



- **Write state to files.** A decision that lives only in the conversation is gone. Files persist.
- **Work in chunks.** Five to ten turns with one objective, then start fresh. Your agent at turn 30 is noticeably worse than at turn 3.
- **Compact deliberately.** `/compact` summarizes and continues; better still, have it write progress to disk first.

Source: A Series on Claude Code by Paul Goldsmith-Pinkham.

Housekeeping

- **No homework, no grading.** Nothing is submitted.
- **Everything ships:** every prompt as it was run, every transcript, every plan – and the failures.
- **No code is provided** – not a downloader, not a parser, not a test. You build all of it with the agent.

`github.com/xuganchen/BootCamp-Introduction-to-Claude-Code`

Part 2. Building a private credit dataset from raw SEC filings

Why private credit, and why BDCs

- Private credit is the **fastest-growing corner of asset management**.
- **BDC filings are its only public window.** A BDC lends to middle-market borrowers and must disclose every position, every quarter.
- **The filing grades your work.** The positions foot to the balance sheet, so you can check the agent against an audited number.

A real AM dataset, with a ground truth to check it against, free and public.

Investment position data from raw SEC filings

- A position-by-position table of **every loan the fund holds**: borrower, industry, lien seniority, spread, maturity, cost, fair value.
- Filed quarterly inside the 10-K and 10-Q, back roughly two decades.
- Example – ARCC's 2026 Q2 10-Q: **24.8 MB, 275 HTML tables, ∼6 million tokens**.

You cannot paste this into a chat window.

Company (1)	Business Description	Investment (18)	Coupon (3)	Reference (7)	Spread (3)	Acquisition Date	Maturity Date	Shares/Units	Principal	Amortized Cost	Fair Value
Software and Services											
ACP Avenu Midco LLC (13)	Provider of technology solutions and hardware to U.S. state and local governments and motor vehicle agencies	First lien senior secured revolving loan	8.65%	SOFR (Q)	5.00%	04/2025	10/2029		$ 0.9	$ 0.9	$ 0.9
		First lien senior secured loan	8.69%	SOFR (Q)	5.00%	08/2025	10/2029		3.6	3.5	3.6
		First lien senior secured loan	8.69%	SOFR (Q)	5.00%	04/2025	10/2029		13.1	13.0	13.1
										17.4	17.6
Actfy Buyer, Inc. (13)	Software provider of end to end fraud management workflow solutions	First lien senior secured loan	8.39%	SOFR (M)	4.75%	05/2024	05/2031		2.1	2.1	2.1
Activate Holdings (US) Corp. and CrossPoint Capital AS SPV, LP (13)	Provider of software services that support the management and security of computing devices, applications, data, and networks	First lien senior secured loan	8.98%	SOFR (Q)	5.25%	09/2024	07/2030		6.8	6.8	6.8
		First lien senior secured loan	8.98%	SOFR (Q)	5.25%	07/2023	07/2030		41.9	41.9	41.9
		Limited partnership interest				10/2023		9,249,000		10.3	13.2
										59.0	61.9
Adonis Bidco Inc. (13)	Provider of intellectual property management lifecycle software	First lien senior secured loan	9.23%	SOFR (Q)	5.50%	02/2025	02/2032		30.0	30.0	28.5
		First lien senior secured loan	9.48% (2.88% PIK)	SOFR (Q)	5.75%	02/2025	02/2032		238.6	238.6	226.7
										268.6	255.2

Building a private credit dataset from SEC filings

- **Panel A – BDC-quarter**
  - One row per BDC per quarter: fund-level financial statements.
  - NAV, leverage, portfolio yield, portfolio size, non-accruals, etc.
- **Panel B – BDC-quarter-investment**
  - One row per investment position per quarter: deal-level terms.
  - Borrower, loan type, seniority, spread, maturity, cost, fair value, etc.
- **Pick your own BDC** (a large one is easiest), window **2023-01-01 to now**.

Step	What we build	The concept it forces
0	Naive baseline: one prompt	A passing check is only as good as its **scope**
1	Write a spec, then build	**Plan mode**, **sub-agents** for independent verification
2	Generalize across quarters	**Skills**, and the tie-out as a test suite
3	Analysis and report	Analysis as verification; **grounding** in your references

Step 0. The naive baseline

Step	What we build	The concept it forces
0	**Naive baseline: one prompt**	**A passing check is only as good as its scope**
1	Write a spec, then build	**Plan mode**, **sub-agents** for independent verification
2	Generalize across quarters	**Skills**, and the tie-out as a test suite
3	Analysis and report	Analysis as verification; **grounding** in your references

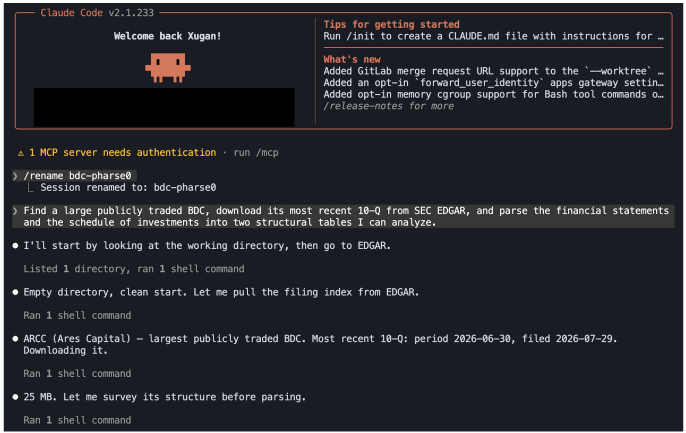
- I will walk through my own output in detail; you do not need to type anything.
- It took $\sim$15 to 20 minutes to finish this one simple prompt.
- I experimented with three different models: Opus 5 at xhigh effort, Opus 5 at medium effort, and Sonnet 5 at medium effort.

Start with the laziest possible prompt

```
Find a large publicly traded BDC, download its most recent 10-Q from SEC EDGAR, and parse
    the financial statements and the schedule of investments into two structural tables I
    can analyze.
```

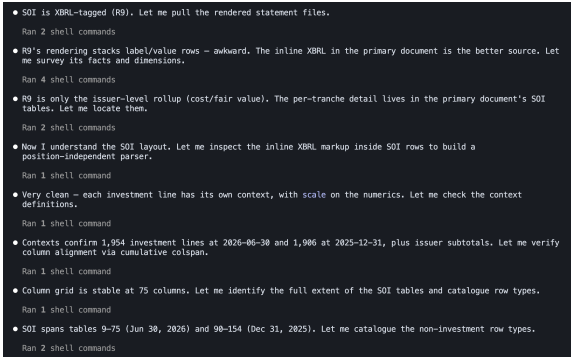
- Not in it: a ticker, a schema, a file format, a column list.
- Not in it: **any word about verifying anything.**

It picked its own target



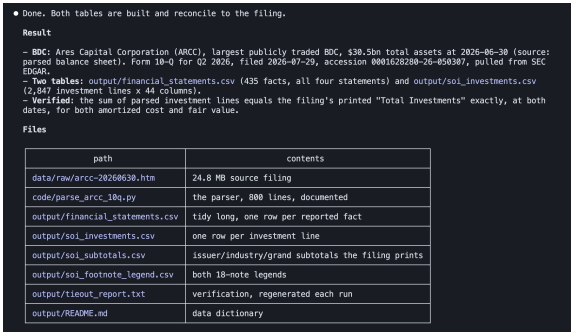
- **Ares Capital (ARCC)**, reasoned from the words “large publicly traded BDC”.
- Its **most recent 10-Q**, quarter ended 2026-06-30: a 24.8 MB inline-XBRL document.

It reads the document before parsing it



- It did not open with a regex. It **surveyed the filing** first: how many tables, where the schedule starts, how the columns repeat.
- Then it chose the **XBRL tags** over the rendered tables – a real judgment call, made unprompted, and the right one for a 2026 filing.
- "**Look before you act**" costs the agent a couple of minutes here, and it is the reason the parse works at all.

What came back, from one sentence



● Done. Both tables are built and reconcile to the filing.

Result

– **BDC:** Ares Capital Corporation (ARCC), largest publicly traded BDC, $30.5bn total assets at 2026-06-30 (source: parsed balance sheet). Form 10-Q for Q2 2026, filed 2026-07-29, accession 0001628280-26-050307, pulled from SEC EDGAR.
– **Two tables:** output/financial_statements.csv (435 facts, all four statements) and output/soi_investments.csv (2,847 investment lines x 44 columns).
– **Verified:** the sum of parsed investment lines equals the filing's printed "Total Investments" exactly, at both dates, for both amortized cost and fair value.

Files

path	contents
data/raw/arcc-20260630.htm	24.8 MB source filing
code/parse_arcc_10q.py	the parser, 800 lines, documented
output/financial_statements.csv	tidy long, one row per reported fact
output/soi_investments.csv	one row per investment line
output/soi_subtotals.csv	issuer/industry/grand subtotals the filing prints
output/soi_footnote_legend.csv	both 10-note legends
output/tieout_report.txt	verification, regenerated each run
output/README.md	data dictionary

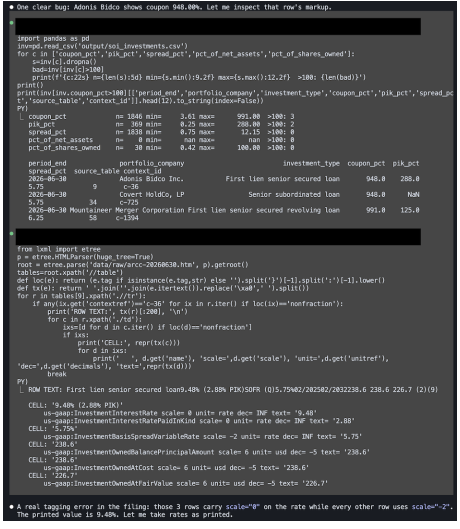
- Two tables: 435 statement facts; **2,847** investment rows (1,439 at 2026-06-30, 1,408 at 2025-12-31).
- Plus a footnote legend, a subtotals file, a README, and a **re-runnable parser** – 803 lines of it.
- Clean CSVs you could hand to an analyst. **Twenty-two minutes, one sentence of instruction.**

Nobody asked for verification. It built one.

Check, each against a number the filing prints	2026-06-30	2025-12-31
A. parsed lines vs printed Total Investments	exact	exact
B. each issuer subtotal vs the lines above it	394/394 agree	380/380 agree
C. schedule total vs balance sheet	within $0.4m	within $0.2m
D. affiliation split vs the three balance sheet buckets	within $0.8m	within $0.8m
E. investments / equity vs the printed % of net assets	211.28 vs 211.32	205.93 vs 205.93

The word “verify” appears **nowhere** in the prompt, and it invented a balance-sheet tie-out anyway. Remember that **it** chose the check, because that is where this ends up.

948.00% coupon – and the filing is wrong, not the agent



- **It also found an obvious bug**: Adonis Bidco shows a coupon of 948.00%.
- Then it dug in: three of the 1,846 coupon facts (Adonis Bidco, Covert HoldCo, Mountaineer Merger) are tagged `scale="0"` where every other rate carries `scale="-2"`. Taking the XBRL value at face value turns a 9.48% coupon into 948%.
- The mis-tagging is **ARCC's, sitting live at SEC**.
- The agent noticed it, worked around it, and wrote it down.

This is genuinely good. Hold that thought.

- One sentence in. A working parser, two clean tables, a documented source-data bug, and a verification report out.
- **Every claim it made about its own output was true.**
- If you stopped here you would think the lesson was “the prompt can be lazy”.

Same prompt, three times

- Identical prompt, identical filing, three configurations:
  - Opus 5 at xhigh effort (max > xhigh > high > medium > low)
  - Opus 5 at medium effort
  - Sonnet 5 at medium effort (Fable 5 > Opus 5 > Sonnet 5)
- Nothing else differs. No extra instruction, no follow-up, no correction.
- This is the experiment sixty of you run every time you all point the same tool at the same task: **same instruction, different machine and model.**

Question: what do you expect to differ – and what do you expect to stay the same?

Three configurations, three successes

Behavior	Opus 5 xhigh, Opus 5 medium, Sonnet 5 medium
Chose the target	ARCC, all three
Chose the representation	inline XBRL tags, not rendered tables, all three
Parsed both periods	yes, all three
Rows at 2026-06-30	1,439, exactly, all three
Invented a balance-sheet tie-out	yes, all three, unasked
Tie-out result	passes both dates, 0.001%, all three

This is robust behavior, not a lucky run.

Three right answers with no shared vocabulary

	Opus 5 xhigh	Opus 5 medium	Sonnet 5 medium
Columns	44	22	26
Company field	`portfolio_company`	`issuer`	`issuer`
Fair value field	`fair_value_usd`	`fair_value`	`fair_value_usd`
Type field	`investment_type`	`instrument`	`investment_type`
Verification shipped as	`tieout_report.txt`	separate script	inside the parser
Parser	803 lines, 1 file	526 lines, 2 files	504 lines, 2 files

There are sixty of you in this room – that is sixty correct, incompatible datasets and no pooled panel.

Count the rows. What does the 1,409 mean?

Run	date	rows	distinct companies	investment type is empty?
Opus 5 xhigh	2026-06-30	1,439	593	0
Opus 5 xhigh	2025-12-31	1,408	580	0
Opus 5 medium	2026-06-30	1,439	605	8
Opus 5 medium	**2025-12-31**	**1,409**	**1,409**	**1,409**
Sonnet 5 medium	2026-06-30	1,439	593	0
Sonnet 5 medium	2025-12-31	1,409	580	1

Why does Opus 5 medium have 1,409 distinct companies, but all of them have an empty investment type field?

Every row became its own borrower

Opus 5 medium, at 2025-12-31:

issuer	investment_type
Automotive Keys Group, LLC and Automotive Keys Investor, LLC, Class A common units	-
Automotive Keys Group, LLC and Automotive Keys Investor, LLC, First lien senior secured loan	-

Opus 5 xhigh, the same rows:

portfolio_company	investment_type
Automotive Keys Group, LLC and Automotive Keys Investor, LLC	Class A common units
Automotive Keys Group, LLC and Automotive Keys Investor, LLC	First lien senior secured loan

- The problem: the instrument type text has been concatenated into the borrower name. It parsed the current period correctly and the comparative period wrongly.
- **And it ties out to the balance sheet exactly, on both dates.** The check was measuring **dollars** while the thing that broke was **structure**.

The broken one is Opus. Sonnet at the same effort is clean.

- **Sonnet 5 at medium effort is clean**: 593 and 580 distinct companies, one null type field, a third of xhigh's authoring budget.
- Model tier and reasoning effort **did not predict correctness** here.
- What the extra effort bought was **coverage**, not correctness: 44 columns vs 26, 14 decoded footnote flags, a subtotals file, a legend file.
- Richer and more auditable. Not more right on the dollars, which all three got right.

"Use the biggest model" is not the lesson. Verification you designed is.

Step 1. Write a spec, then build

Step	What we build	The concept it forces
0	Naive baseline: one prompt	A passing check is only as good as its **scope**
1	**Write a spec, then build**	**Plan mode, sub-agents for independent verification**
2	Generalize across quarters	**Skills**, and the tie-out as a test suite
3	Analysis and report	Analysis as verification; **grounding** in your references

- **This one you type.** Your own BDC, your own filing, your own criteria.
- You write one short file and the agent writes everything else.
- By the end you have a parser that foots to the balance sheet, plus a test suite that proves it.

The fix is not a longer prompt

- Step 0: one line of instruction, three runs, three schemas. All three reported passing their own checks.
- The instinct is to write more instruction. That leaves you exactly where step 0 left you: prose is not checkable.
- **Put the human input where a machine can enforce it.** For example, “zero nulls in the type field, tie-out within 0.1 percent” either holds or it does not.

Write down how you will know it worked before you write down what to build.

Write down how you will know it worked, first

```
# BDC Dataset

* Goal: build a private credit BDC dataset fron SEC filings for the most recent filing of one large BDC.
* Deliverable:
    * BDC-quarter panel: one row per BDC per quarter, fund-level financial statements.
    * BDC-quarter-investment panel: one row per investment position per quarter, deal-level terms.
    * Two consistent panel schemas.
* Acceptance criteria:
    * BDC-quarter panel:
        * Each row must be a BDC-quarter observation.
        * These fields are never null: BDC, CIK, period end, filing date.
        * total liabilities = net assets = total assets, or other basic accounting rules.
    * BDC-quarter-investment panel:
        * Each row must be an investment position.
        * These fields are never null: BDC, period end, borrower, investment type, amount, fair value.
        * The sum of extracted fair value equals total investment in the filing, within 0.1%.
        * Unique borrower names should be fewer than rows, because a BDC holds multiple loans per borrowers.
    * The units should be consistent across rows and tables.
    * If any check fails, exit non-zero and write no output file.
* Folder structure
    * README.md
    * code: filename should be in order, such as "bdc_01_XXX.py", "bdc_09_utils.py", etc.
    * data:
        * raw: organized raw files.
        * interim: temporary processing files.
    * output: final output.
    * note: other misc notes.
```

Create an empty project folder and write your first document, `plan_v0.md`. It answers one question: **what must be true of the output for me to believe it?**

- Objective and scope
- Deliverables and folder structure
- **Acceptance criteria**

Example of plan_v0.md

Objective and scope:

```
* Goal: build a private credit BDC dataset from SEC filings for the most recent filing of
    one large BDC.
```

Deliverables:

```
* Deliverable:
    * BDC-quarter panel: one row per BDC per quarter, fund-level financial statements.
    * BDC-quarter-investment panel: one row per investment position per quarter,
    deal-level terms.
    * Two consistent panel schemas.
```

Example of plan_v0.md

Folder structure:

```
* Folder structure
    * README.md
    * code: filename should be in order, such as "bdc_01_XXX.py", "bdc_09_utils.py", etc.
    * data:
        * raw: organized raw files.
        * interim: temporary processing files.
    * output: final output.
    * note: other misc notes.
```

Example of plan_v0.md

Acceptance criteria:

```
* BDC-quarter panel:
    * Each row must be a BDC-quarter observation.
    * These fields are never null: BDC, CIK, period end, filing date.
    * total liabilities + net assets = total assets, or other basic accounting rules.
* BDC-quarter-investment panel:
    * Each row must be an investment position.
    * These fields are never null: BDC, period end, borrower, investment type, amount,
    fair value.
    * The sum of extracted fair value equals total investment in the filing, within 0.1%.
    * Unique borrower names should be fewer than rows, because a BDC holds multiple loans
    per borrowers.
* The units should be consistent across rows and tables.
* If any check fails, exit non-zero and write no output file.
```

The prompt

You write `plan_v0.md` yourself. Then, note the two constraints packed into one sentence:

```
Only work on current folder. Based on @plan_v0.md, propose a working plan in @plan_v1.md.
```

- `Only work on current folder` bounds the working directory.
- `propose ... in @plan_v1.md` makes the plan a **file you can edit**, not a message you skim.

26 lines in. 210 lines out

BDC Dataset - Working Plan (v1)

Expands `plan_v0.md` into an executable build plan. Scope: one BDC, one filing (most recent 10-Q or 10-K), two panels, hard-gated verification.

1. Decisions to lock before coding

Item	Proposed default	Why
Target BDC	ARCC (Ares Capital Corp)	Largest listed BDC, longest filing history, cleanest Schedule of Investments (SOI) layout
Filing	Most recent 10-Q; fall back to 10-K if the latest period end is annual	Latest SOI plus a balance sheet in the same document
CIK resolution	Look up at runtime from `https://www.sec.gov/files/company_tickers.json`	Do not hardcode a CIK that has not been verified
Language / stack	Python 3.11+, `requests`, `lxml`, `pandas`, `pytest`	No credentials, no paid data
Units	All money fields stored in USD (float, dollars), not thousands or millions	Filings mix thousands and millions across tables; normalize once at parse time and record the source scale
Output format	CSV plus a Parquet twin; a JSON run manifest	CSV for inspection, Parquet for types

Open question for the user: single BDC only, or should the code be written so the BDC is a parameter from day one? The plan below assumes **parameterized, run for one BDC** (same effort, no lock-in).

- It came back with 2 schemas, 16 verification checks, 12 named traps, a 7-step build order, and a table of decisions to lock before coding.
- Because the plan is a file, you can edit one line of it and hand it back.

The plan gave verification gates, and it pushed back on my spec

● plan_v1.md written. Three things it adds beyond v0:

– **A verification gate that can actually catch a wrong parse.** v0's internal checks (assets = liabilities + net assets, fair value tie-out) all pass if the whole table is off by a factor of 1000. v1 adds an independent cross-check against EDGAR's XBRL companyfacts endpoint, plus an explicit source_scale field, so scale errors fail instead of tying out.
– **Named traps with handlers and fixtures.** Ten of them (SOI subtotal rows, split tables, footnote-glued numbers, inherited borrower names, equity rows with no principal). The subtotal-row trap is the likely cause of the first tie-out failure, and the build order expects step 3 to fail before step 4 fixes it.
– **One schema relaxation.** v0 requires amount never-null on the investment panel; equity and warrant positions have no principal. v1 renames it principal_amount and scopes the rule to debt-type rows.

Two open items for you:

Item	Default assumed	Change it?
Target BDC	ARCC	Any large listed BDC works
Parameterization	BDC is a runtime parameter, run once for one BDC	Could hardcode, same effort

CIK is resolved at runtime from company_tickers.json rather than hardcoded, since I have not verified any CIK against the live endpoint.

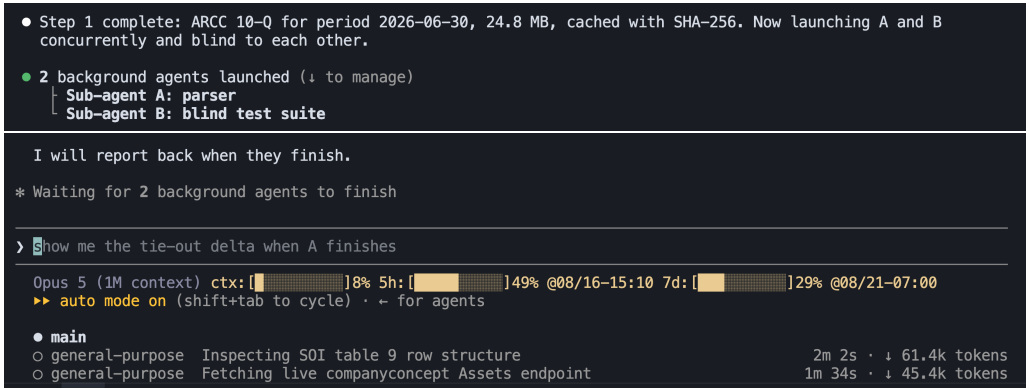
Run it with sub-agents

```
Execute @plan_v1.md. Sub-agent A writes the parser; sub-agent B writes the test from the
    criteria, never seeing the parser.
```

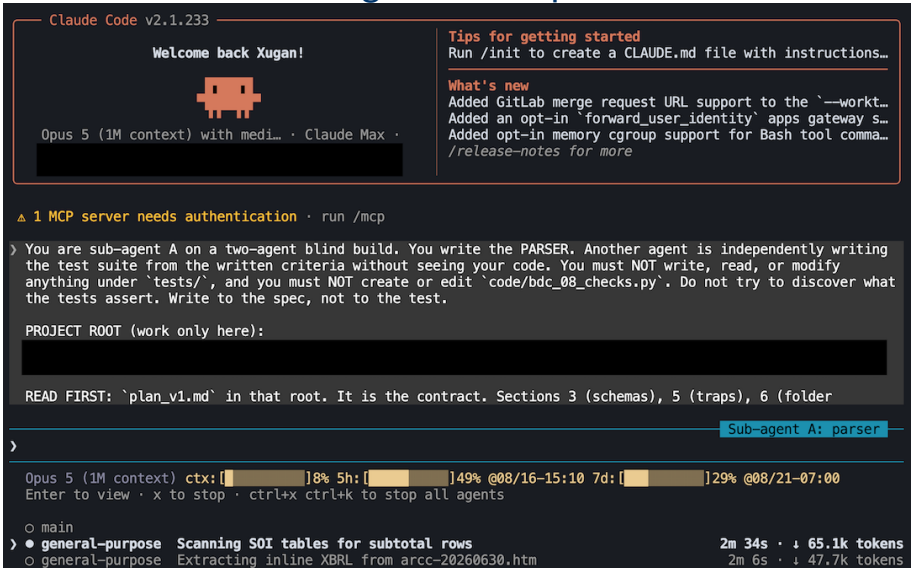
- A sub-agent is a **fresh context** given one task. It does not see the main conversation, and it disappears when the task is done.
- That is what makes B's test independent – but only because the criteria live in a file. If they lived in the chat, B would be reading A's homework.

Never let the thing that wrote the answer also write the check on the answer.

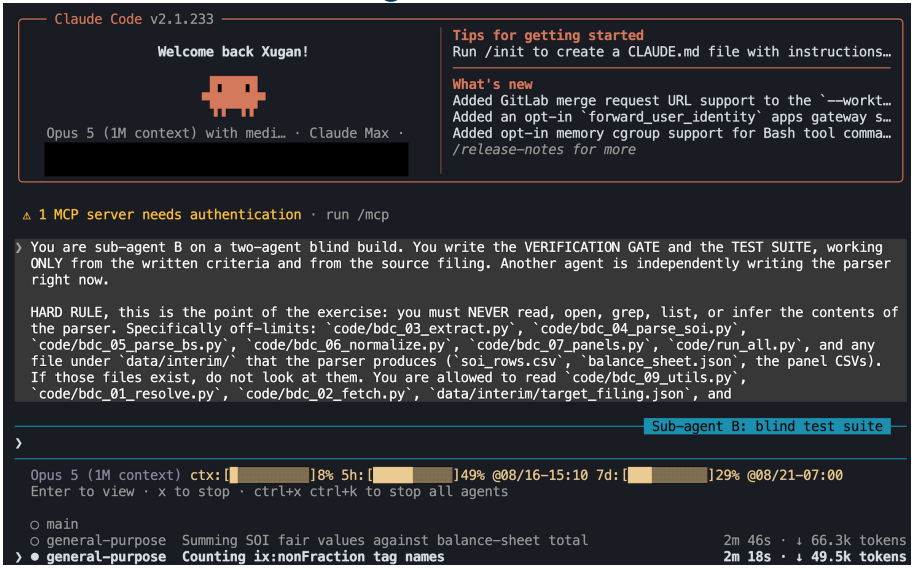
Run it with sub-agents



Sub-Agent A: the parser



Sub-Agent B: the test



Sub-Agent B: the test

● Agent "Sub-agent B: blind test suite" finished · 14m 44s

● Sub-agent B has finished. Sub-agent A is still running, so these numbers are B's report and my own independent verification is pending.

B confirmed it never read the parser. It touched parser output only through `pandas.read_csv` on the two panel CSVs, which is what the gate is supposed to do.

Ground truth, derived three independent ways (rendered balance-sheet text, the filing's own inline XBRL facts, and the companyfacts API):

Field	2026-06-30	2025-12-31
Total investments at FV	29,349,000,000	29,485,000,000
Total assets	30,498,000,000	31,235,000,000
Total liabilities	16,607,000,000	16,917,000,000
Net assets	13,891,000,000	14,318,000,000
NAV per share	19.35	19.94

The filing reports in millions, confirming the per-table scale concern was real. The comparative date was read from the section heading and cross-confirmed by inline XBRL contexts, not inferred.

Organized because the spec said so

- `code/` the pipeline, `tests/` the checks, `output/` the deliverable.
- `note/` is the one folder that was not in plan_v0.md, and it is the most useful: `trap_log.md`, `parsing_decisions.md`, `coverage.md`.
- **None of this is the agent being tidy.** It is the folder structure that was in plan_v0.md.

What it bought, and what it did not

- What it bought:
  - A structured and staged pipeline: 2,757 lines, 10 scripts
  - An independent test suite: 1,067 lines, reusable next quarter
  - Criteria that catch the step 0 break: type never null, borrowers fewer than rows
  - A gate that writes nothing on failure: on any failed check it exits non-zero and writes no output file.
- What it did not buy:
  - A better answer: the output is almost identical to step 0's
  - Speed: 28 min against 15 to 23 min

This works for one filing. There are 88.

- You now have a parser that foots to the balance sheet, plus a test suite that proves it, on **one** 10-Q.
- ARCC has **88 filings** on EDGAR, back to 2004.
- Run this parser, unchanged, across all 88: **5 pass**.

Step 2. Generalize across quarters

Step	What we build	The concept it forces
0	Naive baseline: one prompt	A passing check is only as good as its **scope**
1	Write a spec, then build	**Plan mode**, **sub-agents** for independent verification
2	**Generalize across quarters**	**Skills, and the tie-out as a test suite**
3	Analysis and report	Analysis as verification; **grounding** in your references

- Your parser is proven on **one** filing. A dataset is many.
  - Today: generalize across quarters.
  - Another dimension: generalize across BDCs.
- You package what you learned as a **skill**, then sweep the whole history.
- The deliverable of this step also includes a **coverage statement** that says what the panel does and does not contain.

A script computes. A skill carries the judgment.

- Wrong definition: "a script has the implementation, a skill has the prose." Follow it and you rewrite 2,757 working lines for every new filer.

A skill is the map of (1) what is reusable, (2) what is not, and (3) how to tell the difference on an input it has not seen.

- It **points at** the pipeline and carries the boundary – the thing that does not survive in code and has to be generalized.
- Plus: how to detect which representation a filing uses, the acceptance criteria, the traps with their signatures, and what it must **never** do to make a check pass.

Prompt: package the seam, with a repair loop

```
Read code/, note/trap_log.md and note/parsing_decisions.md.

First, tell me which parts of this pipeline are filer-agnostic and which encode something specific
    to this BDC. Quote the lines.

Then write .claude/skills/bdc-soi-parse/SKILL.md so that an agent facing a BDC or a quarter we have
    never parsed knows: what to reuse unchanged, what to extend and how, how to detect which
    representation a filing uses, the acceptance criteria, and each trap with the signature that
    detects it. It must also say how to repair itself, and what it must never do in order to make
    a check pass.

Do not paste the engine into the skill. Point at it.
```

The skill

● Written to .claude/skills/bdc-soi-parse/SKILL.md (255 lines). Structure:

Section	Contents
1	Reuse-unchanged table: 12 mechanisms with file:line, no code pasted
2	Extend-here table: 15 shared-config tables mapped to the symptom that sends you there, plus the two structural extensions (borrower-on-own-row, control-status sectioning)
3	Representation detection: 15 probes keyed to extract.json / soi_rows.csv fields
4	plan_v0.md acceptance criteria verbatim, the one documented relaxation, and which of the 16 checks matter most on an unfamiliar filing
5	All 12 traps plus the two real bugs, each with a detection signature, handler location, and whether ARCC exercised it
6	Self-repair: 6-step failure protocol, an 11-item never-do list, and the trap-log append template
7	Command sequence for orienting on a new filing

Two things I flagged rather than smoothed over:

- **Trap 4 (parenthesised negatives) is untested against real data.** The handler exists; the ARCC filing never exercised it. The skill marks it and names it as the likely first entry to move to "fired".
- **A silent-wrong-but-green path exists:** if the non-accrual footnote legend is not found, bdc_03_extract.py:336 logs a warning, is_non_accrual is False for every row, and no check catches it. The skill calls this out in the detection table.

Prompt: parameterize, then sweep

```
/bdc-soi-parse ARCC prior 10-Q (2026-03-31)
```

● ARCC 10-Q, period end 2026-03-31, accession 0001628280-26-027688: **16 pass, 0 fail, 0 warn, 0 skip,** promoted to `output/`.

– **No parser, vocabulary, or tolerance change was needed.** Every shared table in the skill's section 2 already matched: SOI headers, balance-sheet labels, type vocabulary, rate canon, scale words, XBRL tags.
– **One extension, and it was a selection gap, not a representation difference:** the pipeline could not target a non-latest filing at all.
– Tie-out 0.001017% (tolerance 0.1%); against the SOI's own printed grand total, **0.000000%.**

The extension

`pick_filing` sorted by period end and returned `rows[0]`, so only the newest 10-Q/10-K was reachable. Added to the shared resolver, serving every filer and quarter:

Change	File
`period_end` / `form` selectors, exposed as `--period-end` / `--form`	`code/bdc_01_resolve.py:26-53`
`main(argv=None)`	`code/bdc_01_resolve.py:86`
Selectors forwarded to stage 01 only; gate called as `main([])` so its argparse never sees them	`code/run_all.py:41-49,78,95-112`

Exact selector, not nearest-match: an unreported period fails with the list of periods that exist; two filings sharing a period end fail asking for `--form`. No accession, CIK, or index hardcoded, and the no-selector default is unchanged.

Prompt: parameterize, then sweep

```
/bdc-soi-parse every 10-K and 10-Q ARCC has filed with a period end on or after 2023-01-01. Work
    oldest to newest so failures cluster visibly.
Then assemble all successful filings into a single BDC-quarter panel and check that (cik,
    period_end) is unique. Report what you find before fixing it.
```

- I ran it three times; each time it found new failures and fixed some of them.
- Pipeline grew 2,757 to 4,009 lines (+45%); `trap_log.md` reached 491 lines.
- **50 of 88 filings (57%)**, 37,197 positions; **30 of 34** from 2018 onward; **14 of 14** from 2023 onward.

Pass	In panel	Note
Baseline (step 1 parser, unchanged)	5 of 88	
After first sweep and repair	22 of 88	Went backwards in time, oldest first
After another pass, asking it to try again	43 of 88	Reached 2005; the recent hole untouched
Focused on post-2018, newest first	**50 of 88**	The recent hole closed

What you already trust must not move

Reference filing, 2026-06-30	After step 1	After step 2
Positions	1,439	1,439
Distinct borrowers	593	593
Sum of fair value (USD)	29,349,300,000	29,349,300,000

- Byte-identical through **45 filings** of edits. It kept a saved copy and diffed after every change.
- It also recorded a bug it **introduced** during the backfill, and how that surfaced.

When you extend a model, make sure the thing you already trust does not move.

Step 3. Analysis and report

Step	What we build	The concept it forces
0	Naive baseline: one prompt	A passing check is only as good as its **scope**
1	Write a spec, then build	**Plan mode**, **sub-agents** for independent verification
2	Generalize across quarters	**Skills**, and the tie-out as a test suite
3	**Analysis and report**	**Analysis as verification; grounding in your references**

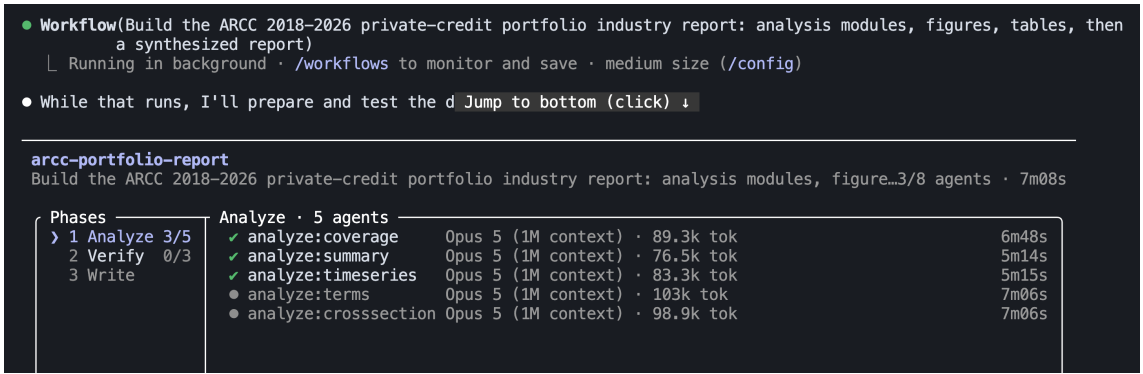
- After Step 2, we have a panel ready for analysis.
- One prompt, one turn, no follow-ups, no debugging.
- And using the data turns out to be **a check of its own**.

The prompt

```
ultracode
Based on the 2018-2026 data in output/panel/bdc_quarter.csv and
    output/panel/bdc_quarter_investment.csv, produce an industry report on this portfolio. Write
    the code to analysis and output everything to output/analysis. The analysis report style
    follows the documents in reference, and should be in both .docx and .pdf, with figures and
    tables. It should at least include sample coverage, summary statistics, time series, deal
    terms, and cross-section. Be specific about percentages and periods. Every figure and table
    states the period it covers and the share of rows it is computed on.
```

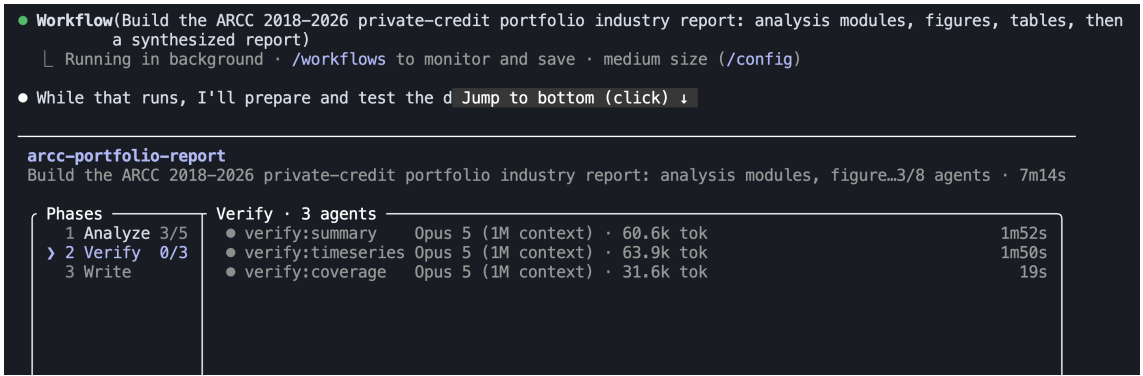
Ultracode

Ultracode is dynamic workflows orchestrate many subagents, skills, and tools, so that it can plan and execute complex tasks.



Ultracode

Ultracode is dynamic workflows orchestrate many subagents, skills, and tools, so that it can plan and execute complex tasks.



Style in `reference/`

- Prompt: **"the analysis report style follows the documents in reference"**.
- `reference/` holds three industry report PDFs I downloaded. No template, no rules – just the documents.
- **What transferred:** exhibit numbering, a source line under every exhibit, a "Finding." sentence opening each section, a limitations section.
- **What did not:** any claim or figure from them. Their conventions, your position.

Further usage

- **Build an index** once the corpus is too big to attach every time: queried rather than pasted, and reusable. Part 4.
- **Turn your own style into a skill**, so the next report starts from your conventions.

One prompt, 53 pages

● Verified and complete. Every executive-summary number reproduces exactly against the raw panels on my own independent recomputation.

Deliverables

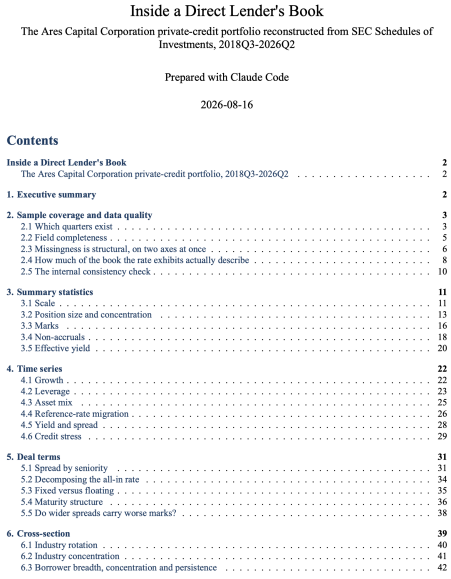
File	Size
output/analysis/ARCC_Private_Credit_Portfolio_Report.pdf	5.3 MB, 53 pages
output/analysis/ARCC_Private_Credit_Portfolio_Report.docx	5.4 MB, 338 paragraphs, 25 tables, 32 embedded images
output/analysis/report.md	source, 175 KB, 0 non-ASCII characters
output/analysis/figures/ tables/ facts/	32 PNG, 24 CSV + 24 MD, 56 JSON fact sidecars
code/analysis/	common.py (scaffold), m1–m5 modules, build_report.py

What the report finds (2018Q3–2026Q2, 30 quarters, 31,067 position-quarter rows, ARCC only)

- Fair value $11.220bn to $29.349bn, +161.6% (13.21%/yr over 7.75 yrs); net assets +89.9% (8.63%/yr), so debt/net assets went 0.622x to 1.135x, peak 1.278x in 2022Q4.
- First lien 44.1% to 59.1% of fair value, second lien 29.9% to 4.4%, equity 9.0% to 15.4%. Senior secured combined actually fell, 74.0% to 63.5%.
- All-in rate on the debt book: 7.82% trough 2021Q4, 12.20% peak 2023Q3, 9.49% in 2026Q2, while FV-weighted spread compressed 707bps to 570bps. LIBOR-to-SOFR crossover in a single quarter, 2023Q2.
- Non-accruals 5.06% of cost peak (2020Q3), 2.38% of cost / 1.38% of fair value in 2026Q2; non-accrual debt carried at 0.5839x cost.
- Borrowers 313 to 593; top-10 share 25.55% to 22.43%. Software and Services +9.09pp to 21.98%; Healthcare Equipment and Services −10.42pp to 9.86%.

- Report: 53 pages, 21,006 words, 32 figures and 48 tables.
- Analysis is not the cheap part: 4,086 lines of code. The previous pipeline: 4,009 lines.

The report



Inside a Direct Lender's Book

The Ares Capital Corporation private-credit portfolio reconstructed from SEC Schedules of Investments, 2018Q3-2026Q2

Prepared with Claude Code

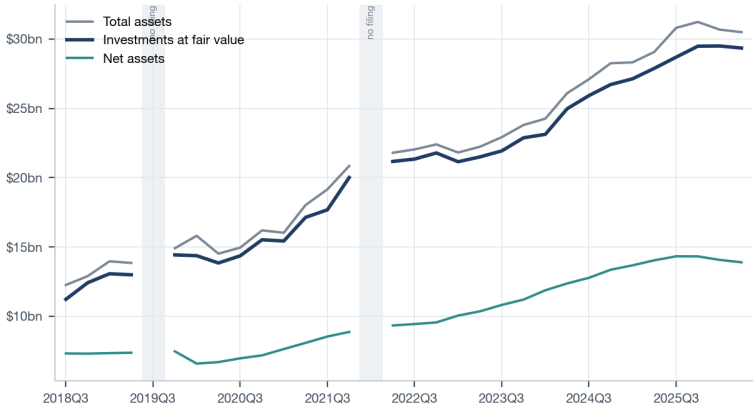
2026-08-16

Contents

- Eight sections, among them: executive summary, coverage, summary statistics, time series, deal terms, cross-section, limitations.
- Every exhibit carries **its period and its coverage**, because the prompt said so and the code enforced it.

Example exhibit: the time series of balance sheet growth

Balance sheet in levels, quarter by quarter
Investments at fair value grew from $11.2bn to $29.3bn (13.2% per year over 7.75 years)



Period covered: 2018Q3-2026Q2. Computed on 30 of 30 quarter-panel rows (100.0% of the window panel). The panel holds 30 of the 32 calendar quarters in the window: 2019Q3 and 2022Q1 are absent and are shown as breaks; no value is interpolated or bridged across them. The growth rate is an endpoint-to-endpoint compound rate over the 7.75 years from 2018-09-30 to 2026-06-30, not an average of quarterly growth.
Source: author's calculations on the ARCC BDC panel built from SEC EDGAR 10-K/10-Q filings.

The analysis found a bug that previous checks passed over

The report, in its own section 4.6, **unprompted**:

> *"The non-accrual series is drawn on 27 of the 30 quarters: 2022Q4, 2023Q4 and 2024Q4 return exactly zero parsed flags while neighbouring quarters carry 13 to 22, which is a* ***footnote-marker parse failure and not a clean book****."*

- **Why no check saw it.** `is_non_accrual` is a boolean, so **no tie-out reaches it** – and the column is 0.00% null: fully populated, and in three quarters uniformly wrong.

Analysis can be viewed as another verification layer.

Part 3. More useful concepts

Working tips

1. Plan before acting.
2. Be specific.
3. Always have a backup.
4. Manage your context window.
5. Always have a way to verify outputs.
6. Ask the LLM to take notes.
7. Be replicable: 22 minutes to author, 2 seconds to run.
8. Stay organized with your code, data, and notes.

AI safety: honestly, you will grant most permissions

- Every step today asked for file writes, shell access and network access. It is unrealistic to approve every single permission every time.
- The best you can do is to allow-list broadly and stop reading.
- Which is tip 3 again, for a different reason: **always have a backup.**
- Mine is GitHub for code and Dropbox for data, and my own agents run on **a separate Mac Mini** – a machine with limited access to my data that I can wipe.
- The real question is not “what may it do” but **“what can it reach”**.

Dev containers are the real alternative

- Cost: Docker, a config file, and a slower first run. Windows needs WSL2.
- Isolation is about **what is reachable**. It says nothing about what the agent does with what it can reach.
- See “#7 Permissions, Sandboxes, and Autonomous Agents” in *A Series on Claude Code*, by Paul Goldsmith-Pinkham.

Part 4. Advanced usage examples

Each of these adds something the project could not show

Demo	Axis it adds	Where Part 2 stopped
GBrain	Retrieval that is *indexed*	Step 3 attached sample docs by hand
TravelTab + PetTab	The pipeline becomes *software*	We shipped a report, not a system
NanoClaw	The agent has an *address*	You were always at the terminal

Nothing here is a new tool. It is the same agent, moved one step along one axis.

GBrain: personal-knowledge brain

You ask:

> "What do I need to know before my meeting with Alice tomorrow?"

Most personal-knowledge tools give you back a list of pages. Something like:

```
1. people/alice — Alice runs engineering at Acme...
2. meetings/2026-03-15-alice-q1 — Q1 product review with Alice...
3. meetings/2026-01-08-acme-kickoff — Kickoff meeting with Acme team
4. customers/acme — Acme is a series-B fintech we work with...
5. notes/2026-04-22 — Quick chat with Alice about pricing...
```

GBrain gives you back the answer, with sources:

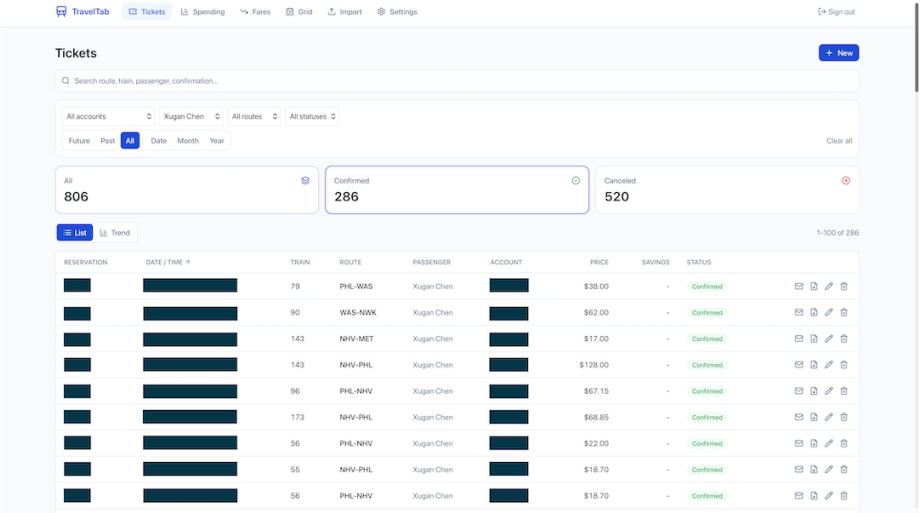
```
Alice runs engineering at Acme (a series-B fintech). You last spoke
on April 22 in a quick pricing chat. Three things are still open
from that conversation:

1. She owes you the security review for the new tier
   (deadline was May 1; no update since).
2. You committed to pricing for a 500-seat tier
   (you sent it April 25; no response yet).
3. She mentioned they're hiring a CISO; you said you'd intro
   someone from your network.

Heads up: nothing's been added to the brain about Alice or Acme
since April 22, six weeks ago. She may have replied through email
or Slack DM, channels the brain doesn't see. Worth asking her to
catch up before assuming any of this is still current.
```

- **GBrain** indexes a corpus you own – meeting notes, summaries, Notion – and exposes it to the agent as a tool it can query.
- The key idea: it (1) searches the corpus, (2) reads the hits for you, and (3) writes the answer.
- `github.com/garrytan/gbrain`

TravelTab: personal-database management

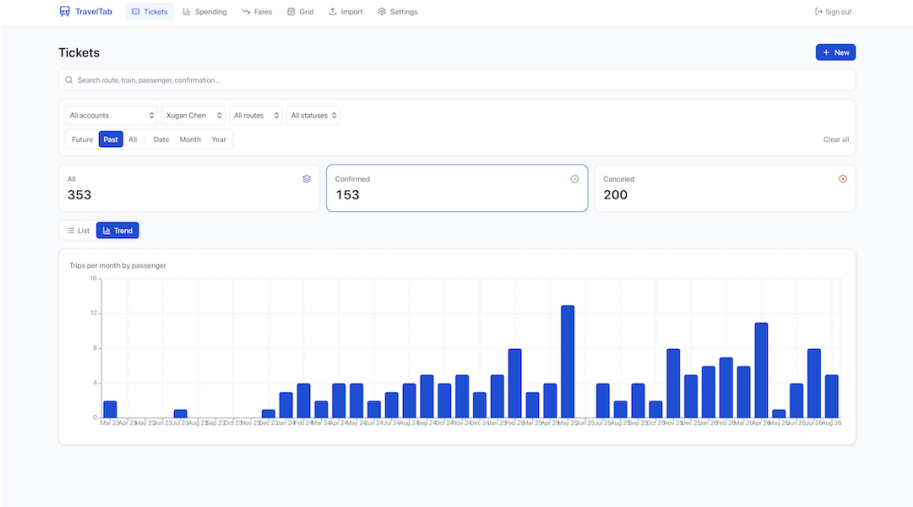
- I travel between New Haven and Philadelphia constantly, and Amtrak is the only workable way to do it. The app is painful, it does not talk to my calendar, **the ticket arrives once by email and sometimes it is gone**, and the pricing is dynamic.
- So I built **TravelTab**, a personal Amtrak ticket tracker:
  1. **Ingests tickets automatically from my email.**
  2. **Exports structured records into a database**: one row per passenger per ticket.
  3. **Syncs into my calendar**, so a trip is something I can actually manage.
  4. **Fare watch** that checks prices and alerts me.

TravelTab: personal-database management



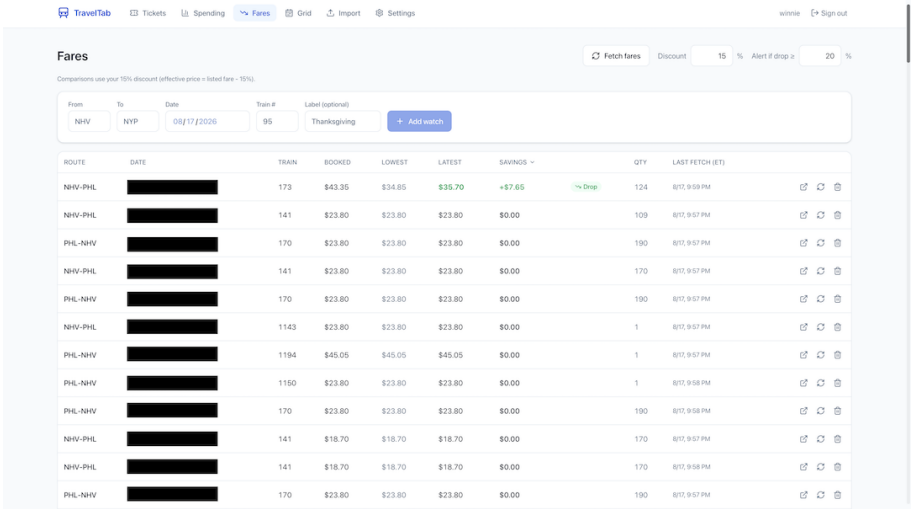
Every row here arrived as **an email**. Messy documents in, structured records out.

TravelTab: personal-database management



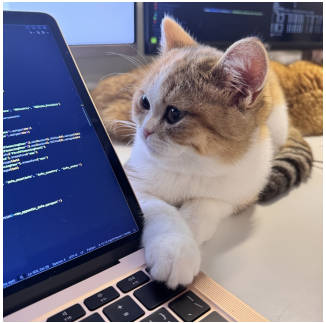
This is what the database bought: it tells me how often I have been travelling.

And it keeps running when you are not watching



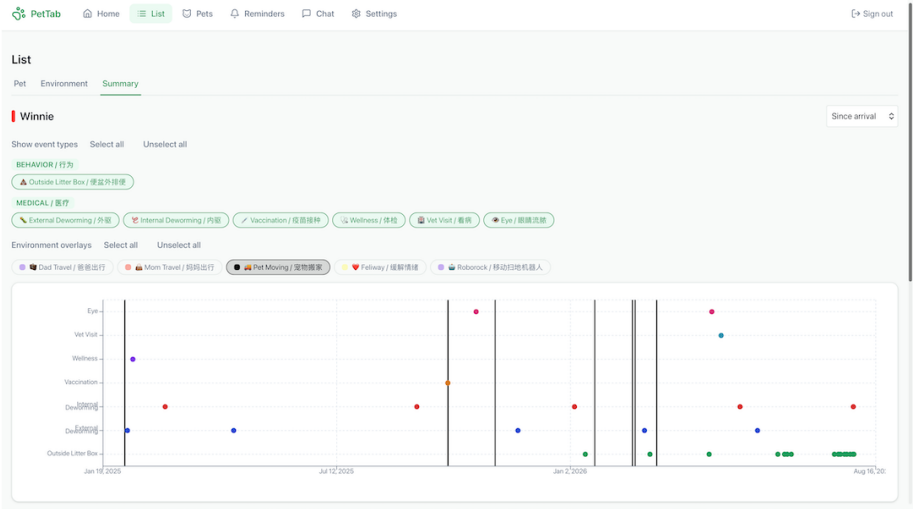
Fare watchlist that checks prices on a schedule and alerts me on a threshold.

PetTab: another domain, same stack

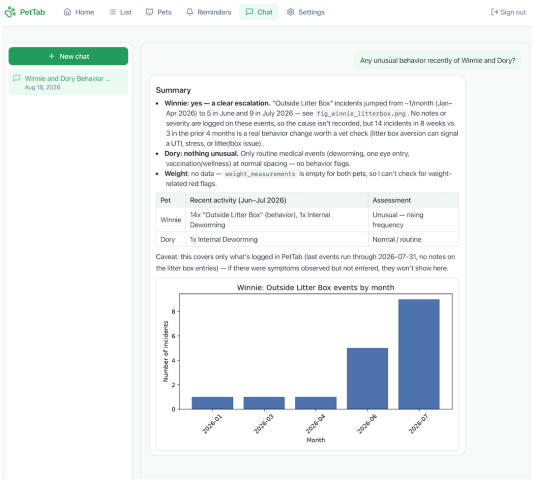


- Background: I have two cats, Winnie and Dory, and I want to track their health and behaviour. Existing tools are too complicated, and I want full control over the data.
- So I built **PetTab**, a pet-health logger built on TravelTab's stack.

PetTab: visualizing the data



PetTab: a chat agent inside the product



- The key difference: the agent is embedded in the product itself.
- PetTab's Insights chat runs the `claude` CLI over a **read-only snapshot** of the database, so that it can explore the recent behavior and health history of the cats, and give me suggestions on what to do next.
- Each chat gets its own working directory holding that read-only copy, restricted tools, a lock that serialises turns, and admin-only access.

An agent with an address

- **NanoClaw**: Claude in a Linux container, reached from WhatsApp, Slack, Telegram or email.
- Every minute so far had you at a terminal, driving. Same tool – reached by sending it a message.
- A personal-scale project: one process and a handful of files, running in a Linux container.
- `github.com/nanocoai/nanoclaw`

Where to go next

- Try to use it in your own projects and daily life as much as possible.
- Brainstorm your ideas, and then build them.

Other useful resources:

- Agent Skills with Anthropic – DeepLearning.AI short course
- Finance skill plugins by Anthropic
- A collection of resources on AI tools: `www.xuganchen.com/public-goods`
- Alternatively, ask AI to build it for you.

Course materials and every prompt run today:
`github.com/xuganchen/BootCamp-Introduction-to-Claude-Code`